Getting Started with MVBot-V3

This worksheet guides you through the initial setup and operation of the MVBot-V3 robotic platform.

1. OpenMV cam operation

Install and launch the OpenMV IDE, then plug the USB-c cable into the robot's rear. Open camera.py, click **Connect** and **Run** at the bottom-left corner of the IDE.

Step 1: focus the camera lens

You will see the live stream from the camera, and you need to make sure it focuses on the floor 15cm in front of the robot. To focus the camera lens:

- Open the top hatch of the robot carefully
- Rotate the camera lens until the image is sharpest at ~12cm from the lens
- Close the hatch and lock it with the screws

Step 2: tune in for a colour target

The robot tracks objects by colour. You will define the colour thresholds in __main__ inside camera.py.

- In the IDE, open the **Colour Space** tab and set it to **LAB Colour Space**
- Hold a coloured object in front of the lens and observe how the LAB values change
- Go to **Tools -- Machine Vision – Threshold Editor (image from buffer)**: adjust all sliders until you can just make up the target. Copy the colour definition vector at the bottom and paste into your script.
- Save and re-run the script. Your target colour should now be detected with a **bounding box**

Step 3: verify the coordinate rotation

The camera sits under a periscope that rotates when panning. This rotation physically rotates the project image on the sensor in the roll axis, so we correct the blob coordinates mathematically rather than rotating the entire image (which is slow).

Follow these steps to verify the correction is working:

Find the periscope range:

- Rotate the periscope fully in one direction until it hits the mechanical end-stop
- Pull the periscope up, reinsert it, and rotate fully the other way to find the other end-stop
- Confirm the camera is looking at the symmetric angle on both sides

Test the coordinate correction:

- Centre the periscope and place your colour target ~10–15cm in front of the camera
- In __main__, set angle = 45 and rotate_image = False, then run the script
- You should see a **bounding box** around the blob and a **cross** inside it. Now force the periscope rotation: you should see the cross shifting diagonally at 45° while the bounding box shits horizontally. This confirms the coordinate rotation is working

Sanity check with image rotation:

- Set rotate_image = True and re-run the script
- The **cross should now stay inside the bounding box of the blob**, while image is rotated.
- You will also notice the live stream becomes sluggish. This is why we only rotate coordinates during normal robot operation, not the full image.

2. OpenMV servo operation

Once the camera setup is complete, close camera.py and open servos.py.

Step 1: verify the servo mapping

- Connect the lithium battery inside the robot and set the rear switch to **Mobile Mode**
- With the robot connected to the IDE, run servos.py. You should see the following:
    1. **Left wheel forward – left wheel backward**
    2. **Right wheel forward – right wheel backward**
    3. **Pan servo jerking left 45° -- right 45° -- left 90° -- right 90°**

- If the sequence and behaviour is correct, your servo mapping is confirmed.

Step 2: find the servo neural point

Continuous rotation servo motors can drift slightly even when commanded to stop. You need to find the exact zero-speed setting for each servo.

- In servos.py, modify __main__ to extend the **zero speed state** and re-run the script
- If any wheel is still drifting, adjust the following values until the wheels are completely still:
  self.left_zero — left wheel neutral point
  self.right_zero — right wheel neutral point
- Verify the periscope is pointing **straight forward** at its neutral position; adjust self.pan_angle_corr as needed.
- Finally, adjust self.half_pan_us to ensure the full pan rotation covers exactly **180 degrees**

Once both the camera and servo setups are complete, you are ready to proceed to the **first autonomous foraging exercise**.